CONFIDENTIAL

PRAXIS Project Plan

Self-Improving Knowledge Loop for Claude Code Agents • 9-Day Focused Sprint

Team Members & Leadership Roles

Three Gauntlet AI Fellows collaborating on a production-grade capstone. Each member leads one distinct, high-impact aspect of the project to enable authentic technical interview storytelling.

Matthew Daw	Monica Peters	Dominic Antonelli
ML & Knowledge Pipeline Lead Ingestion, learning moment detection (ML classifier), LLM distillation, consolidation/dedup/scoring, knowledge graph, provenance. ***Interview claim:*** *"I led the ML-powered distillation engine that turns raw JSONL logs into scored, deduplicated, human-approved knowledge with full provenance."*	**Dashboard & Human Gate Lead** React review dashboard, human approval workflow (proposed→suggested→active), contradiction resolution UI, credibility metrics viz, injection controls. ***Interview claim:*** *"I designed and built the human approval dashboard that enforces quality gates and makes knowledge promotion transparent and measurable."*	**Architecture, Eval & Integration Lead** System design, eval harness (fixed tasks + metrics), GitHub hook/PR automation, Python tooling, deployment, live demo & compounding curve proof. ***Interview claim:*** *"I architected the eval harness and integration layer that rigorously proves PRAXIS delivers ≥50% fewer corrections with compounding gains."*

Project Overview

PRAXIS turns Claude Code's session history into a queryable **Knowledge Graph** — the project's source of truth and primary deliverable — so the agent stops re-learning the same things. As Claude Code works across ~20 projects, its session messages stream into a DynamoDB store. A *learning moment* — a pull request, or an insight a user inserts via a Claude tool — fires a trigger; the related session and GitHub context is then distilled by an LLM into structured candidates. Candidates are clustered, deduped, scored, and decayed, with a human stepping in only to approve 100%-credible insertions or resolve contradictions. Approved knowledge lands in the Knowledge Graph. Future sessions pull just-in-time context from it

through a **"get context" tool call** — a Python function that reads the session history, the current codebase, and the graph, and returns a focused block of context to inject into the conversation. A Knowledge Graph Dashboard gives visibility and optional editing, and a rigorous eval harness replays known PR/ticket tasks cold vs. knowledge-injected to quantify the compounding gains.

System Architecture

Claude Code streams its session messages into a DynamoDB log store. A learning moment — a pull request, or a user-inserted insight — fires the **Trigger**, which gathers the related session and GitHub context and hands it to LLM distillation. Distilled candidates are consolidated, scored, and gated (humans intervene only for fully-credible insertions or contradictions) before entering the **Knowledge Graph**. Claude Code reads the graph back through the "get context" tool call, and the Eval Harness measures cold vs. knowledge-injected runs.

PRAXIS Architecture: Self-Improving Knowledge Loop for AI Coding Agents

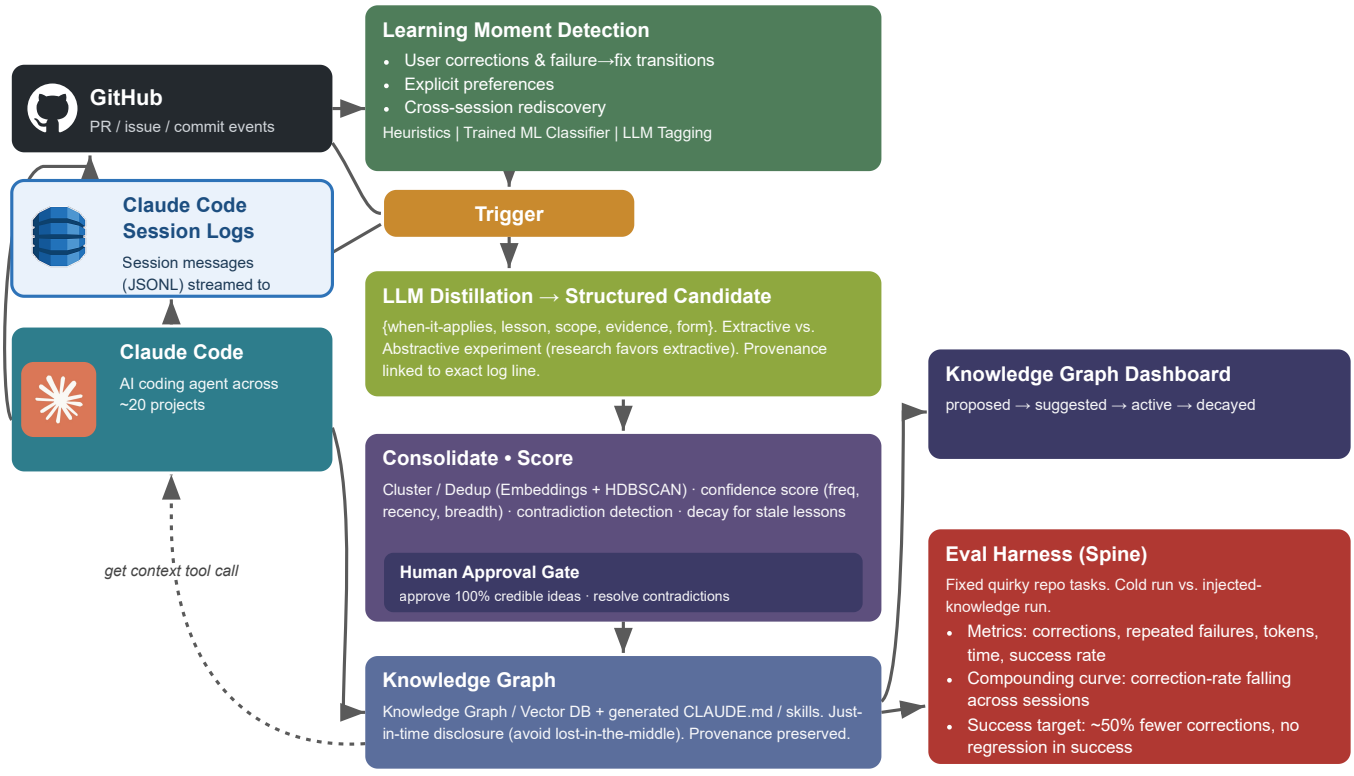

Figure 1: PRAXIS End-to-End Architecture — Claude Code sessions and pull requests become a verified Knowledge Graph, read back into future sessions via a get-context tool call, with measurable compounding gains.

Target Outcome & Success Metrics

- **MVP delivered:** learning-moment detection → distillation → cluster/dedup/confidence → human gate (credible insertions & contradictions only) → Knowledge Graph + get-context tool → Knowledge Graph Dashboard → eval harness showing quantified improvement.
- **Primary success criterion:** ≥50% reduction in user corrections on benchmark tasks vs. cold runs, with no regression in task success rate, plus visible compounding curve across sessions.
- Each team member owns one pillar end-to-end and can demonstrate it live in technical interviews.

High-Level 9-Day Timeline

Days 1–2: Project Plan, Foundation & Design | Days 3–5: Parallel Core Build (ML Pipeline + Dashboard + Evals) | Days 6–7: Integration & Human Gate | Day 8: Eval Harness & Measurement | Day 9 - 10: Demo Polish, Documentation & Final Runs

Detailed 9-Day Work Schedule

All three contributors work in parallel streams with daily syncs. Primary ownership ensures clear leadership claims while cross-support builds team cohesion.

Day	Focus & Milestones	Matthew (ML/KG)	Monica (Dashboard)	Dominic (Arch/Eval)
1	Kickoff, repo setup, log exploration, architecture finalization	Explore sample JSONL logs; define episode segmentation heuristics	Dashboard wireframes & tech stack (React + state mgmt)	Eval harness skeleton + fixed quirky repo tasks; GitHub hook design
2	Design review, data contracts, first prototypes	Prototype learning-moment detector (pull requests + manual-insight tool)	Build review dashboard shell + candidate list view	Define eval metrics & cold-run baseline script
3	Core pipeline build (parallel)	Full distillation pipeline + structured candidate output + provenance	Candidate detail view + confidence score UI components	Python tooling (reader, distiller wrapper) + basic injection
4	ML & consolidation focus	Embeddings + HDBSCAN dedup/cluster; contradiction detection logic	Human gate workflow UI (proposed → suggested → active)	Eval harness: scripted PR/ticket recreation runs (throwaway artifacts, metrics only)
5	Scoring, decay, dashboard polish	Confidence scoring (freq/recency/breadth) + decay rules	Contradiction resolution interface + credibility metrics viz	Eval harness expansion: token/time tracking + basic dashboard
6	Integration sprint	End-to-end pipeline wiring + knowledge graph stub	Dashboard ↔ backend API integration + approval actions	Full eval harness + cold vs injected comparison runner
7	Human gate + injection complete	Knowledge Graph build + get-context tool (session history + codebase + graph → injected context)	Full human approval flow + provenance display in UI	Eval harness recreates PRs/tickets per replay run (deleted after scoring); demo data prep

8	Measurement & refinement	Run batch evals; analyze failure modes; tune thresholds	Dashboard polish + edge-case handling in review flow	Full compounding curve measurement; identify demo quirks
9	Demo, docs, handoff	Final pipeline tuning; knowledge quality report, Practice Presentation	Dashboard demo-ready; user flow video capture	Live demo script + side-by-side before/after; final docs & repo handoff